Java w IntelliJ IDEA - konwersje typów i obliczenia

BLOK 2 - rzutowanie, promocja typów i przepełnienie

Punkt startowy: W Lab 2 były już **int, double, char, String, zmienne i operatory**. Nie powtarzamy ich definicji. Teraz badamy, **jak Java określa typ wyrażenia, kiedy traci dane i co się dzieje przy wartościach granicznych**.

1. Literały — tylko nowe reguły zapisu

Znasz już literały, np. **12** i **1.75**. W tym temacie potrzebujemy również przedrostków systemów liczbowych i sufiksów, które wpływają na typ wartości. Prefiks zmienia zapis liczby w kodzie, nie wartość liczbową w pamięci.

```
int dec = 42;
int bin = 0b101010;
int hex = 0x2A;
int oct = 052;
long big = 8_000_000_000L;
float ratio = 1.5F;
```

0b oznacza zapis binarny, **0x** szesnastkowy, a liczba zaczynająca się od **0** może być literałem ósemkowym. **L** oznacza long, **F** — float. Podkreślenia w liczbie ułatwiają czytanie. Porównaj wynik w **LiteralBases.java**.

2. Konwersja rozszerzająca — automatyczna, ale nie zawsze dokładna

```
int points = 150;
long a = points;        // 150
double b = a;          // 150.0
long huge = 9_007_199_254_740_993L;
double rounded = huge; // możliwa utrata dokładności
```

Z **int** do **long** przechodzimy bez jawnego rzutowania. Podobnie **long** można przypisać do **double**, ale duże wartości całkowite nie muszą być reprezentowane dokładnie. Automatyczna konwersja nie oznacza pełnego zachowania wszystkich cyfr.

3. Konwersja zawężająca — jawny rzut

Przy przejściu z **double** do **int** wymagane jest rzutowanie. Dla zwykłych, mieszczących się w zakresie wartości część ułamkowa jest **odrzucana w kierunku zera**, a nie zaokrąglana.

```
double price = 19.99;
int whole = (int) price; // 19
int negative = (int) -8.8; // -8
```

Przy konwersjach do węższych typów całkowitych może też zmienić się sama wartość. **byte** mieści się w zakresie od -128 do 127.

```
int code = 130;
byte result = (byte) code;
System.out.println(result);  // -126
```

Po konwersji z **int** do **byte** zachowywane są najmłodsze 8 bitów wyniku. Przećwicz wartości 127, 128, 255, 256 i -129, najpierw przewidując rezultat.

4. Typ obliczenia ustala się przed przypisaniem

Najważniejsza reguła: Typ zmiennej po lewej stronie znaku **= nie zmienia automatycznie** tego, jak zostało wykonane działanie po prawej. Najpierw obliczane jest wyrażenie, dopiero później jego wynik trafia do zmiennej.

```
int apples = 5;
int people = 2;
double a = apples / people;            // 2.0
double b = (double) apples / people;   // 2.5
double c = (double) (apples / people); // 2.0
```

Wynik **apples / people** jest całkowity, bo oba operandy mają typ **int**. Rzutowanie **przed** dzieleniem zmienia typ działania; rzutowanie dopiero gotowego wyniku nie odzyska utraconej części ułamkowej. Uruchom **DivisionPrecision.java**.

5. Promocja arytmetyczna wyrażeń

W operacjach arytmetycznych wartości **byte**, **short** i **char** są zwykle promowane do **int**. Przy mieszaniu **int**, **long**, **float** i **double** wynik uzyskuje typ wynikający z reguł promocji operandów.

```
byte x = 10, y = 20;
int sum = x + y;         // 30; wynik wyrażenia: int
// byte bad = x + y;    // błąd kompilacji
int code = (int)'A' + 1; // 66
char next = (char) code; // 'B'
```

Zauważ, że tu nie przypominamy, czym jest **char**. Badamy zmianę typu **wyrażenia**. Pomocny plik: **Example17NumericPromotion.java**.

6. Przepełnienie int — poprawna składnia, błędna liczba

```
long wrong = 100_000 * 100_000;
long correct = 100_000L * 100_000;
System.out.println(wrong);   // 1410065408
System.out.println(correct); // 10000000000
```

W pierwszym wierszu mnożenie **int * int** przepełnia zakres **int**, zanim wynik zostanie przypisany do **long**. Sufiks **L** przy jednym argumencie powoduje wykonanie mnożenia w typie **long**. Zobacz **IntegerOverflow.java**.

Przepełnienie standardowej arytmetyki całkowitej **int** w Javie nie zgłasza automatycznie wyjątku. Gdy potrzebujesz sprawdzania zakresu, poznasz później np. **Math.multiplyExact**, które sygnalizuje przepełnienie wyjątkiem.

7. Zamiana String na liczbę to parsowanie, nie rzutowanie

```
String input = "120";
int value = Integer.parseInt(input);
System.out.println(value + 30); // 150
// Integer.parseInt("120zl") -> NumberFormatException
```

Zapis **(int) input** nie zamieni tekstu na liczbę. Program musi odczytać zapis liczby. Błędny format tekstu może spowodować **NumberFormatException**.

8. Szybka tabela porównawcza

Przykład	Co trzeba zauważyć
(int) 8.8	Odrzucenie części ułamkowej → 8.
(int) -8.8	Odrzucenie w kierunku zera → -8.
8 / 3	Dzielenie całkowite → 2.
8.0 / 3	Wynik zmiennoprzecinkowy → 2.6666…
(byte) 130	Wynik zawężenia → -126.
long r = 100000 * 100000	Obliczenie najpierw w int; możliwe przepełnienie.

9. Skrócone przypisanie i niejawna konwersja

Operatory **+=**, **-=** i podobne mogą zachowywać się inaczej niż pełne wyrażenie zapisane z użyciem **=**. W szczególności przy typach o małym zakresie w zapisie skróconym występuje konwersja wyniku do typu zmiennej. Nie oznacza to, że obliczenie jest zawsze bezpieczne.

```
byte level = 120;
// level = level + 10;  // błąd kompilacji: wynik ma typ int
level += 10;           // dopuszczalne, wynik: -126
System.out.println(level);
```

W obu przypadkach dodawanie działa na wartościach promowanych do **int**. Przy zapisie skróconym wynik zostaje następnie zawężony do **byte**. Dlatego po dodaniu 10 do 120

powstaje wartość -126. Zapis += nie chroni przed przekroczeniem zakresu.

To zachowanie jest jednym z powodów, dla których podczas ćwiczeń z konwersjami warto analizować nie tylko wartość wyniku, ale także typ każdego podwyrażenia. Porównaj ten przykład ze zwykłym **int** i z **long**.

10. Granica dokładności double i porównania

Typ **double** oferuje duży zakres wartości, ale skończoną liczbę cyfr znaczących. Niektóre bardzo duże liczby całkowite są zaokrąglane podczas konwersji do double. Dwie różne liczby typu long mogą po konwersji stać się tą samą wartością zmiennoprzecinkową.

```
long first = 9_007_199_254_740_992L;
long second = 9_007_199_254_740_993L;
double a = first;
double b = second;
System.out.println(first == second); // false
System.out.println(a == b);          // true
```

Równość dwóch wartości **double** w tym doświadczeniu nie oznacza, że liczby wyjściowe były równe. Oznacza, że utracono informację o różnicy między nimi. Nie próbuj odzyskiwać jej rzutowaniem z double z powrotem na long. W danych wymagających pełnej dokładności trzeba świadomie dobierać reprezentację.

Podobny problem dotyczy reprezentacji wielu ułamków dziesiętnych. Wartość **0.1** nie ma skończonego zapisu dwójkowego; dlatego dodawanie i porównywanie wartości zmiennoprzecinkowych wymaga ostrożności. To inne zjawisko niż zwykłe odcięcie części ułamkowej przy **(int)**.

11. Wartości poza zakresem przy konwersji double do int

Przy rzutowaniu wartości **double** do **int** ważny jest nie tylko znak, ale również zakres i wartości specjalne. Jeżeli skończona liczba jest zbyt duża, wynik konwersji zatrzymuje się na odpowiedniej granicy zakresu int. Wartość **NaN** zostaje zamieniona na zero. To nie jest zachowanie zaokrąglania matematycznego.

```
System.out.println((int) 8.9);
System.out.println((int) -8.9);
System.out.println((int) Double.POSITIVE_INFINITY);
System.out.println((int) Double.NaN);
```

Spodziewane kolejne wyniki to **8**, **-8**, **2147483647** i **0**. Weryfikuj je oddzielnie, aby nie pomylić reguł dla zwykłych liczb, wartości spoza zakresu i wartości specjalnych. Przy projektowaniu praktycznych programów takie konwersje zwykle wymagają wcześniejszej walidacji.

12. Metody sprawdzające przepełnienie

Zwykłe **+**, **-** i ***** na int nie sygnalizują automatycznie przepełnienia. W bibliotece standardowej są jednak metody, które celowo zgłaszają wyjątek, gdy wynik nie mieści się w zakresie typu. Ich działanie będzie można pogłębić podczas tematu o wyjątkach.

```
int a = 2_000_000_000;
int b = 1_000_000_000;
// int c = a + b; // przepełnienie, bez wyjątku
// Math.addExact(a, b); // ArithmeticException
```

W tym temacie wystarczy umieć rozpoznać, kiedy obliczenie w typie **int** nie może dać poprawnego wyniku. Rozwiązaniem może być wybór **long** już dla operandów albo zastosowanie metody z kontrolą zakresu. Nie należy jednak traktować późniejszego przypisania wyniku do long jako zabezpieczenia.

13. Programy Java — kolejność pracy

Plik	Cel
LiteralBases.java	Różne zapisy tej samej liczby.
Example15Widening.java	Konwersja rozszerzająca.
Example16Narrowing.java	Utrata części ułamkowej i zawężenie do byte.
Example17NumericPromotion.java	Typ wynikowego wyrażenia.
DivisionPrecision.java	Rzut przed i po dzieleniu.
IntegerOverflow.java	Przepełnienie i poprawne mnożenie long.
CastingEdgeCases.java	Wartości graniczne i konwersje.
Example14Literals.java	Opcjonalne przypomnienie z Lab 2.
LiteralPractice.java	Zadanie samodzielne: zapisy liczb i sufiksy.
CastingPractice.java	Zadanie samodzielne: rzutowania.

Przykłady Java — kod zgodny z projektem IntelliJ

Najpierw przewidź wynik każdego programu. Potem uruchom właściwy plik z katalogu **src/block2extra** i porównaj wynik z komentarzem. Kod w PDF i w projekcie jest taki sam.

LiteralBases.java

Różne zapisy liczby 42.

```
package block2extra;

public class LiteralBases {
    public static void main(String[] args) {
        int decimal = 42;
        int binary = 0b101010;
        int octal = 052;
        int hex = 0x2A;
        System.out.println(decimal + " " + binary + " " + octal + " " + hex);
    }
}
```

Wynik: 42 42 42 42

Example15Widening.java

Konwersja rozszerzająca bez jawnych rzutowań.

```
package block2extra;

public class Example15Widening {
    public static void main(String[] args) {
        int points = 150;
        long pointsLong = points;
        double pointsDouble = pointsLong;

        char grade = 'A';
        int unicode = grade;

        System.out.println(pointsLong);
        System.out.println(pointsDouble);
        System.out.println(unicode);
    }
}
```

Wynik: 150, 150.0, 65 — każda wartość w osobnym wierszu

Example16Narrowing.java

Utrata części ułamkowej i zmiana wartości po rzutowaniu do byte.

```
package block2extra;

public class Example16Narrowing {
    public static void main(String[] args) {
        double price = 19.99;
        int whole = (int) price;

        int value = 130;
        byte small = (byte) value;

        System.out.println("double -> int: " + whole);
        System.out.println("int 130 -> byte: " + small);
    }
}
```

Wynik: double -> int: 19; int 130 -> byte: -126

Example17NumericPromotion.java

Typ obliczenia i promocja byte oraz char.

```
package block2extra;

public class Example17NumericPromotion {
    public static void main(String[] args) {
        byte a = 10;
        byte b = 20;
        int sum = a + b; // wynik nie jest byte, tylko int

        int count = 3;
        double price = 4.5;
        double total = count * price;

        char letter = 'A';
        int nextCode = letter + 1;

        System.out.println(sum);
        System.out.println(total);
        System.out.println(nextCode + " -> " + (char) nextCode);
    }
}
```

Wynik: 30; 13.5; 66 -> B

DivisionPrecision.java

Porównaj rzutowanie przed i po wykonaniu dzielenia.

```
package block2extra;

/*
 * PRZYKŁAD – DZIELENIE I RZUTOWANIE
 * CEL: sprawdź typ wyrażenia, zanim zostanie przypisane do double.
 *
 * OCZEKIWANY WYNIK:
 * Bez rzutowania: 2.0
 * Rzut przed dzieleniem: 2.5
 * Rzut po dzieleniu: 2.0
 *
 * PYTANIE: dlaczego rzutowanie gotowego wyniku nie odzyskuje ułamka?
 */
public class DivisionPrecision {
    public static void main(String[] args) {
        int apples = 5;
        int people = 2;

        double withoutCast = apples / people;
        double before = (double) apples / people;
        double after = (double) (apples / people);

        System.out.println("Bez rzutowania: " + withoutCast);
        System.out.println("Rzut przed dzieleniem: " + before);
        System.out.println("Rzut po dzieleniu: " + after);
    }
}
```

Wynik: 2.0, 2.5, 2.0

IntegerOverflow.java

Przepełnienie int i mnożenie z operandem long.

```
package block2extra;

/*
 * PRZYKŁAD – PRZEPEŁNIENIE PRZY MNOŻENIU
 * CEL: sprawdź, że long po lewej stronie nie zmienia obliczenia int * int.
 *
 * OCZEKIWANY WYNIK:
 * Obliczenie int: 1410065408
 * Obliczenie long: 10000000000
 *
 * PYTANIE: dlaczego dopisanie L do jednego operandu zmienia wynik?
 */
public class IntegerOverflow {
    public static void main(String[] args) {
        long wrong = 100_000 * 100_000;
        long correct = 100_000L * 100_000;

        System.out.println("Obliczenie int: " + wrong);
        System.out.println("Obliczenie long: " + correct);
    }
}
```

Wynik: 1410065408 oraz 10000000000

CastingEdgeCases.java

Sprawdź wartości graniczne rzutowań.

```
package block2extra;

public class CastingEdgeCases {
    public static void main(String[] args) {
        double x = 123.99;
        int truncated = (int) x;
        int large = 130;
        byte wrapped = (byte) large;
        System.out.println(truncated);
        System.out.println(wrapped);
    }
}
```

Wynik: 123 i -126

14. Ćwiczenia bez gotowych odpowiedzi

Zadanie 1. Przewidź wynik **(int) 8.8**, **(int) -8.8**, **8 / 3** i **8.0 / 3**. Wskaż typ każdego wyniku, a dopiero potem sprawdź w IntelliJ.

Zadanie 2. Zmodyfikuj przykład zawężania: sprawdź wartości 127, 128, 255, 256 i -129 po rzutowaniu do **byte**. Wyjaśnij powstałe wartości.

Zadanie 3. Dla **int apples = 5; int people = 2;** uzyskaj średnią 2.5 bez zmieniania typów obu zmiennych. Zademonstruj też błędną kolejność rzutowania.

Zadanie 4. Porównaj wynik **200_000 * 200_000** z **(long) 200_000 * 200_000**. Dlaczego przypisanie do **long** nie naprawia pierwszego wyniku?

Zadanie 5. Zamień tekst **"120"** na liczbę i dodaj 30. Następnie sprawdź tekst **"120zł"**. Nazwij wyjątek i wyjaśnij, dlaczego zwykły rzut nie działa.